

132 Pattern Approaches

Yes, **132 Pattern** is a very good problem because it can be solved in multiple ways, and each approach teaches a different DSA pattern.

The condition is:

```
i < j < k
nums[i] < nums[k] < nums[j]
```

Think of the three values as:

```
1      3      2
small big  middle
↑      ↑      ↑
nums[i] nums[j] nums[k]
```

For example:

```
[3, 1, 4, 2]
  ↑      ↑
  1      2
        ↓
        4
1 < 2 < 4
```

So [1, 4, 2] is a 132.

Approach 1 — Brute Force

The most direct approach is to try every possible triplet.

We need:

```
C++
for i
  for j > i
    for k > j
      check nums[i] < nums[k] < nums[j]
```

Code

```
C++
class Solution {
public:
    bool find132pattern(vector<int>& nums) {
        int n = nums.size();

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                for (int k = j + 1; k < n; k++) {
                    if (nums[i] < nums[k] && nums[k] < nums[j]) {
                        return true;
                    }
                }
            }
        }

        return false;
    }
};
```

Complexity

Time: $O(n^3)$
Space: $O(1)$

For:

$n = 2 * 10^5$

this is obviously impossible.

So we need to optimize.

Approach 2 — Prefix Minimum + Searching for 2

This is the first important optimization.

Remember:

```
nums[i] < nums[k] < nums[j]
```

For every j , we need:

```
1 = some smallest value on the left
3 = nums[j]
2 = some value after j
```

So instead of trying every i , we can maintain the **minimum value before j** .

Define:

```
leftMin[j] = minimum value in nums[0 ... j-1]
```

Then for every j :

```
1 = leftMin[j]
3 = nums[j]
```

Now we only need to find some $nums[k]$ where:

```
leftMin[j] < nums[k] < nums[j]
```

So the problem becomes:

For every j , can I find a number after j lying inside $(leftMin[j], nums[j])$?

Example

```
nums = [3, 1, 4, 2]
```

Prefix minimums:

```
nums:    3   1   4   2
leftMin: -   3   1   1
```

At:

```
j = 2
nums[j] = 4
leftMin = 1
```

We need:

```
1 < nums[k] < 4
```

Looking after j :

```
2
```

Yes:

```
1 < 2 < 4
```

Therefore:

```
[1, 4, 2]
```

How do we search efficiently?

We can maintain the elements after j in a sorted **set**.

For every j :

1. Find minimum on the left.
2. Search in the right side for the smallest value greater than **leftMin**.
3. If that value is smaller than $nums[j]$, we found a pattern.

C++:

```
C++
class Solution {
public:
```

```

bool find132pattern(vector<int>& nums) {
    int n = nums.size();

    if (n < 3)
        return false;

    int leftMin = nums[0];
    set<int> right;

    // Initially all elements except nums[0]
    for (int i = 1; i < n; i++) {
        right.insert(nums[i]);
    }

    for (int j = 1; j < n - 1; j++) {
        // Remove nums[j]
        // because k must be AFTER j
        right.erase(right.find(nums[j]));

        // Find smallest number > leftMin
        auto it = right.upper_bound(leftMin);

        if (it != right.end() && *it < nums[j]) {
            return true;
        }

        leftMin = min(leftMin, nums[j]);
    }

    return false;
}
};

```

Complexity

set operations take:

$O(\log n)$

for each element.

Therefore:

Time: $O(n \log n)$
 Space: $O(n)$

This works for $2 * 10^5$.

Approach 3 — Prefix Minimum + Sorted Array / Binary Search

We can improve the previous idea conceptually.

Instead of a **set**, we can keep the right-side values in sorted order and use binary search.

The important observation remains:

```

1 = minimum on left
3 = nums[j]

Find:
1 < 2 < 3

```

So binary search can find the smallest value greater than 1.

However, maintaining a dynamically sorted array while moving j requires insertion/deletion, which costs $O(n)$ with a normal vector.

Therefore, this approach isn't as convenient as **set**.

You can think of this as:

```

Prefix minimum
+
ordered structure
=
 $O(n \log n)$ 

```

This is useful because it leads us toward the optimal solution.

Approach 4 — Monotonic Stack — Optimal $O(n)$

This is the **standard and best solution**.

This problem has a beautiful trick.

Instead of directly searching for:

```
1 < 2 < 3
```

we scan from **right to left**.

Why?

Because when we're at `nums[i]`, everything to its right can potentially be:

```
3 and 2
```

We want:

```
nums[i] < 2 < 3
```

So:

```
nums[i] = 1
```

and we need to find a valid 2 on the right.

The key idea

Maintain:

```
stack = possible "3" values
```

and:

```
second = largest valid "2" value
```

The variable `second` is extremely important.

Let's use:

```
nums = [3, 1, 4, 2]
```

Scan from right:

```
2
4
1
3
```

Step 1

Current:

```
2
```

Stack:

```
[2]
```

No 2 value found yet.

```
second = -INF
```

Step 2

Current:

```
4
```

We have:

```
4 > 2
```

Therefore:

```
2 < 4
```

So 2 can act as the middle value.

But the stack value 2 can be promoted to our candidate **second**.

```
second = 2
```

Then push 4:

```
stack = [4]
```

Conceptually:

```
2 < 4
```

```
  2
 / \
2   4
```

Actually think:

```
second = 2
stack  = possible 3 values
```

Step 3

Current:

```
1
```

Check:

```
nums[i] < second
```

That means:

```
1 < 2
```

YES!

Therefore:

```
1 < 2 < 4
```

So:

```
[1, 4, 2]
```

is our 132.

Return:

```
true
```

Why do we pop from the stack?

This is the most important part.

Suppose:

```
stack = [5, 7, 10]
```

and current value is:

```
8
```

We see:

```
8 > 10 ? NO
8 > 7  ? YES
```

We pop 7.

Why?

Because:

```
7 < 8
```

Therefore 7 can become our 2.

But we want the **largest possible 2**, because a larger 2 makes it easier for some future 1 to satisfy:

```
1 < 2
```

So:

```
second = max(second, popped value)
```

More precisely, because the stack is maintained decreasingly, the last popped value is the largest popped value.

The stack invariant

The stack is maintained in **decreasing order**.

For example:

```
[10, 7, 4, 2]
```

If current value is:

```
8
```

we pop:

```
2
4
7
```

until:

```
10
```

is greater than 8.

The last popped value is:

```
7
```

Therefore:

```
second = 7
```

Now we have:

```
7 < 8
```

meaning:

```
2 = 7
3 = 8
```

Optimal Code

```
C++
class Solution {
public:
    bool find132pattern(vector<int>& nums) {
        int n = nums.size();
        if (n < 3)
            return false;

        // "second" represents the 2 in 132
        int second = INT_MIN;

        // Stack stores possible 3 values
        stack<int> st;

        // Traverse from right to left
        for (int i = n - 1; i >= 0; i--) {
            // nums[i] can be the "1"
            //
            // If nums[i] < second,
            // then we have:
            //
            // nums[i] < second < some stack value
            //
            // which forms 132.
```

```

    if (nums[i] < second) {
        return true;
    }

    // Find a bigger value that can act as "3"
    // Every popped value can become "2"
    while (!st.empty() && nums[i] > st.top()) {
        second = st.top();
        st.pop();
    }

    // Current value becomes a possible "3"
    st.push(nums[i]);
}

return false;
};

```

Dry Run

Let's take:

```
nums = [-1, 3, 2, 0]
```

Expected:

```
true
```

Start:

```
second = -INF
stack = []
```

0

```
0 < second ? NO
stack = [0]
```

2

```
2 < second ? NO
```

Since:

```
2 > 0
```

pop:

```
second = 0
```

Then:

```
stack = [2]
```

We have:

```
2 = 3
0 = 2
```

3

Check:

```
3 < 0 ? NO
```

Then:

```
3 > 2
```

Pop:

```
second = 2
```

Stack:

```
[3]
```

Now:

```
2 < 3
```

-1

Check:

```
-1 < 2
```

YES.

And we already know:

```
2 < 3
```

Therefore:

```
-1 < 2 < 3
```

Indices are also correct because we're scanning from right to left.

So:

```
[-1, 3, 2]
```

is a 132.

Return:

```
true
```

Why does the algorithm guarantee $i < j < k$?

This is a very important question.

We're traversing:

```
right → left
```

Suppose:

```
nums[i] = 1
```

When we're processing it, everything in the stack / **second** came from positions **after** **i**.

So automatically:

```
i < j < k
```

The right-to-left traversal handles the index constraint for us.

All Approaches Comparison

Approach	Idea	Time	Space
Brute Force	Try every (i, j, k)	$O(n^3)$	$O(1)$
Prefix Min + Set	Left minimum + ordered right values	$O(n \log n)$	$O(n)$
Prefix Min + Binary Search	Search right-side values	roughly $O(n \log n)$ with suitable ordered structure	$O(n)$
Monotonic Stack	Scan right $\rightarrow$ left	$O(n)$	$O(n)$

For the constraints:

```
n <= 2 * 10^5
```

the one you should learn and use is:

```
Monotonic Stack
O(n) time
O(n) space
```


The Intuition You Should Remember

Don't try to memorize the code.

Memorize this transformation:

Original condition

```
nums[i] < nums[k] < nums[j]
```

Think:

```
1 < 2 < 3
```

Now scan from the **right**.

The right side gives us potential:

```
3
```

and whenever a smaller value gets popped from the stack, it becomes a potential:

```
2
```

So maintain:

```
stack → possible 3
second → best 2
```

Then every new number on the left asks:

```
Can I be 1?
nums[i] < second ?
```

If yes:

```
1 < 2 < 3
```

132 pattern found.

The entire algorithm boils down to:

```
C++
for (i = n-1 → 0)
    if (nums[i] < second)
        return true;

    while (stack not empty && nums[i] > stack.top())
        second = stack.top(), pop;

    push(nums[i]);
```

This is one of those problems where understanding **why second represents the 2** is much more important than memorizing the monotonic-stack code.

Today 12:20 PM

Yes — this problem becomes **much easier once you can visually see what the stack and second are doing**.

Let's use the example:

```
nums = [3, 1, 4, 2]
```

We want:

```
1 < 2 < 4
```

where:

```
1 → nums[i]
4 → nums[j]
2 → nums[k]
```

The key idea is:

```

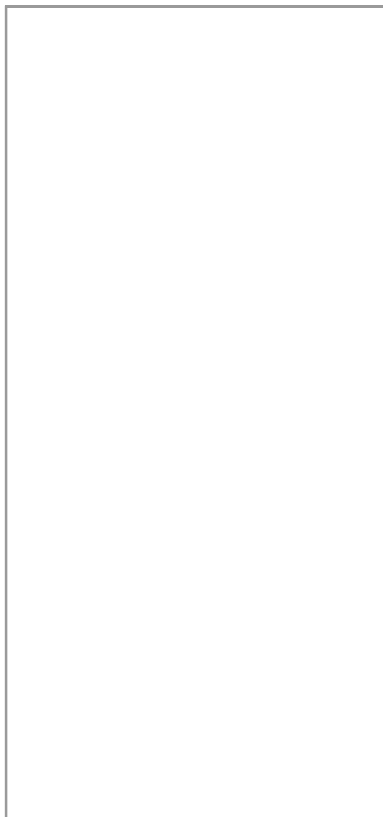
      132
      |
-----
    1  3  2
  
```

```

      ↑   ↑   ↑
nums[i] nums[j] nums[k]

```

We scan **from right** → **left**.



Now let's understand why this works

The most important thing is to stop thinking of the stack as just "some numbers."

We give every variable a job:

```

stack
↓
possible 3

second
↓
possible 2

current nums[i]
↓
possible 1

```

So at any point we're trying to construct:

```

current      second      stack
  ↓          ↓          ↓
  1          <          2          <          3

```

Why do we scan from RIGHT to LEFT?

Suppose we're currently at:

```
nums[i] = 1
```

Everything we've already processed is to its **right**.

Therefore anything stored in:

```

second
stack

```

automatically has an index greater than **i**.

So if we find:

```
nums[i] < second
```

we already know:

```

      i       j       k
      ↓       ↓       ↓
    [1] ... [4] ... [2]

```

Therefore:

```
i < j < k
```

is automatically satisfied.

That's the reason for scanning right → left.

The most important part: Why pop()?

Look at:

```
nums = [3, 1, 4, 2]
```

We start:

```
2
```

Stack:

```
2
```

Now we see 4.

```
4 > 2
```

So:

```
2 ← POP
```

Because:

```
2 < 4
```

we have discovered:

```
2 < 4
```

This means:

```
2 → candidate for "2"
4 → candidate for "3"
```

So:

```
second = 2
```

and:

```
stack = [4]
```

Now the state is:

```

      4
      ↑
    "3"
      |
      2
      ↑
    "2"

second = 2
stack  = [4]

```

Then we encounter:

```
1
```

Check:

```
1 < second
1 < 2
```

YES!

So:

```
1   2   4
↑   ↑   ↑
"1" "2" "3"
```

Therefore:

```
1 < 2 < 4
```

🎯 132 found.

One more important thing

When we do:

```
C++
while (!st.empty() && nums[i] > st.top()) {
    second = st.top();
    st.pop();
}
```

we are basically saying:

"Every stack value smaller than my current value can become the 2, because the current value can act as the 3."

For example:

stack:

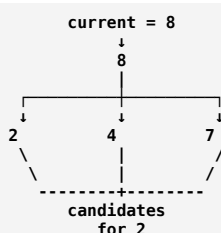
10
7
4
2

current = 8

Since:

```
8 > 2 → pop
8 > 4 → pop
8 > 7 → pop
8 > 10 → stop
```

So:



The last/largest popped value is 7, so:

```
second = 7
```

Now we have:

```
7 < 8
```

meaning:

```
2 < 3
```

We're simply waiting for some future value on the left that satisfies:

1 < 7

If we later encounter:

5

then:

5 < 7 < 8

132 pattern.

The mental picture to remember

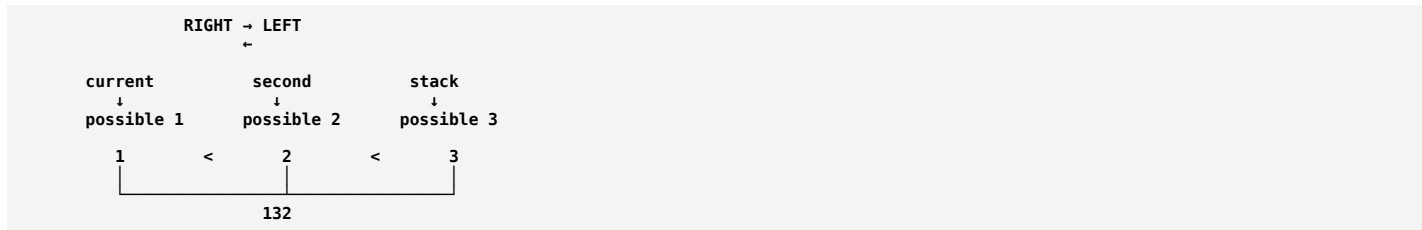
Whenever you see this code:

```
C++
if (nums[i] < second)
    return true;

while (!st.empty() && nums[i] > st.top()) {
    second = st.top();
    st.pop();
}

st.push(nums[i]);
```

visualize:



stack finds the 3 → popping creates the 2 → current element becomes the 1.

That's the entire monotonic-stack trick.

132 PATTERN

Find if there exists $i < j < k$ such that $nums[i] < nums[k] < nums[j]$

Think: $1 < 2 < 3$

$i < j < k$

KEY IDEA

- Scan from **RIGHT** to **LEFT**.
- Stack stores possible "3" values in decreasing order.
- second** stores the best possible "2" (the largest value that is smaller than some "3" seen so far).
- If **current value** < **second**, we found $1 < 2 < 3$.

LEGEND

- Current element (possible 1)
- second (best possible 2)
- Stack (possible 3)
- Action / Explanation

Example: $nums = [3, 1, 4, 2]$

We scan from right to left.

Step	i (index)	Current $nums[i]$	Stack (possible 3)	second (possible 2)	Action	Explanation / Diagram
1	3	2	top 2	-∞	Push 2 into stack	2 is a possible "3"
2	2	4	top 2	-∞	$4 > stack.top() (2)$ → Pop 2 $second = 2$ Push 4	(2) (pop) → 2 second Now we have $2 < 4$ (2 is candidate for "2", 4 is candidate for "3")
3	1	1	top 4	2	Check $1 < second?$ $1 < 2 \rightarrow YES$	$1 < 2 < 4$ 1 < 2 < 4 found! Return true

WHY DOES THIS WORK?

- We are scanning from right to left.
- All elements in stack and in second are to the right of current index i.
- When we pop from stack, the popped value is smaller than current value, so it can be the "2" and current value can be the "3".
- If we find a number smaller than second on the left, it becomes the "1".
- Therefore, indices are automatically in the order $i < j < k$.

FINAL SUMMARY (ALGORITHM)

Initialize: $second = -\infty$, $stack = empty$

For i from $n-1$ down to 0:

- If $nums[i] < second \rightarrow return true$
- While stack not empty and $nums[i] > stack.top()$:
 $second = stack.top()$
 $stack.pop()$
- Push $nums[i]$ into stack

If no pattern found $\rightarrow return false$

So in the whole process we maintain: $current (1) < second (2) < stack \text{ value } (3)$

Whenever $current < second$, we get $1 < 2 < 3 \rightarrow 132 \text{ pattern exists.}$

ChatGPT can make mistakes. Check important info.